

Parallel Logic Expert with Operator Routing

Steven Cleasby-Mayeda
Oregon State University

cleasbys@oregonstate.edu steven.cleasby.mayeda@gmail.com

Abstract

We introduce a logic-augmented transformer pathway that routes token-level representations into fixed fuzzy logical operators (AND/OR/NOT) or straight-through-estimation, so the network learns compositional structure rather than an ensemble of expert networks with shared architecture design. At each backbone layer, the hidden states are projected to a lower dimensional K , Q , which the running logic state queries (cross-attention). The attention matrix is routed, and local windows over either token or gate-channel axes are composed. The final logic state is fused with the backbone features before classification. The proposed architecture augmentation imposes structured, interpretable compositions over token features. A key design dimension is the axis of composition: intra-token compositions (feature axis) compares features within a token, while inter-token compositions (sequence axis) compares features across neighboring tokens. We evaluate these design choices with Llama-3.1-8B under matched parameter budgets over three seeds. On ProofWriter with the rule-base in the input, every variant reaches 0.93–0.94 validation accuracy, and no routed variant beats a parameter-matched No-Gate Control (inter-token $G = 8$: -1.3 pp, intra-token: -0.4 pp); removing cross-attention changes nothing measurable. Routing entropy stays at 97–98% of its maximum and a learned fusion weight shrinks, so the model largely routes uniformly and down-weights the branch. On MNLI, routed and No-Gate variants are also indistinguishable. We report this as a null result for operator routing at this scale, together with the data-pipeline errors that produced the gains reported in the first version of this paper.

Revision note (27 September 2026). The first version (25 March 2026) reported ProofWriter gains for routing. A re-verification found that the ProofWriter loader dropped the rulebase, so every ProofWriter model saw only the question, and that the Baseline pooled the BOS token of a causal model, which made it input-blind. All ProofWriter

and multi-task numbers in this version come from corrected reruns (3 seeds each); MNLI numbers are 3-seed reruns of the original setup. The original values are kept for reference in Table 1 and Figure 2. The verification report, code and per-run logs are in the public repository ([logic/docs/verification](https://github.com/steven-cleasby/logic-docs-verification)).

1. Introduction

Transformers provide strong performance on a wide variety of tasks [12], both chain-of-thought (CoT) prompting and reasoning models can greatly improve performance on reasoning tasks [13, 3]. However, many CoT use N-shot prompting. Additionally, both CoT and reasoning models utilize reasoning by iterative, and extended inference, which requires far more tokens. This increases inference costs for many tasks [13]. Our approach instead introduces a novel architecture for structured computation within the latent space, no additional tokens needed. We will test how this constrained, logic-inspired pathway affects performance on reasoning problems and generalization.

Mixture-of-Experts (MoE) models [5, 10, 4] introduce conditional computation by routing tokens to specialized subnetworks, or weighting the results of several/all experts. Our method introduce conditional computation by routing, weighting tokens to logical operators. In this formulation, the operator routing is learned rather than expert weights. When tied with attention, this method may allow robust routing in the logic space. This space has forced sparsity due to softmax and local compositions (local intra-token or inter-token windows), which may allow emergence of specialized operator-feature connections.

Our activation metrics allows learned logic compositions to be interpreted. Routing entropy and fusion α parameters allow us to measure the utilization of the logic expert. Experiments on logical reasoning benchmarks compare the Baseline, the No-Gate Control network (a baseline with matched parameter count for fair comparison), and the logic expert variants. A core design question is how the local operator windows should be applied in the gate compositions. Our first implementation applies capacity on the channel axis, (neighboring channels on one token). The next design

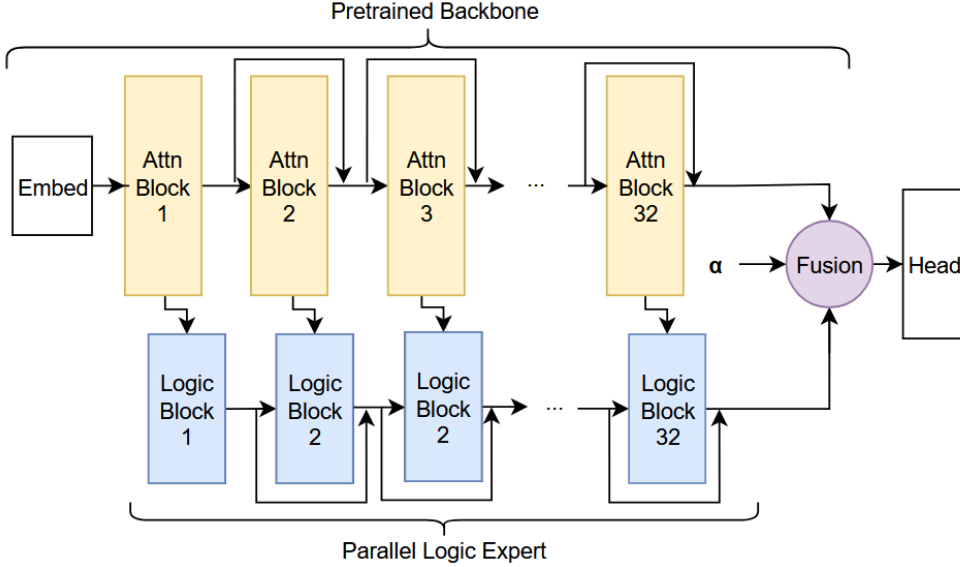


Figure 1. Our architecture, backbone hidden-states feed to logic expert, and fused back with the backbone features before classification

applies capacity on the token axis, each channel compares the same indexed feature on neighboring tokens (channels remain independent). This axis change could affect which feature relationships are learnable; we test whether it affects performance. Figure 1 provides a high-level view of the architecture.

Contributions of this paper are: (1) inter-token compositional routing with operator reductions and residual logic-state updates; (2) controlled, seed-paired ablations over gate count, window axis, fusion α , and cross-attention against a parameter-matched control; (3) a negative result: on ProofWriter and MNLI, operator routing does not improve over the parameter-matched control, and mechanism metrics (near-maximal routing entropy, a shrinking learned α) indicate the branch is barely specialized; and (4) an account of the evaluation errors behind the first version’s positive result.

2. Related Work

Logical and logic-inspired neural architectures span a wide variety of designs, from classic weightless systems to modern differentiable operator networks [1]. Some architectures utilize explicit logical, or logic-inspired differentiable operators [8] as an alternative to dense transformations, and they provide evidence that structured logical computation can be more efficient and interpretable than typical transformers. Our approach builds on this motivation, but integrates operator routing directly into transformer hidden-state rather than standalone units.

Deep Differentiable Logic Gate Networks combine real-valued logical operators with continuous relaxations (differentiable approximations of discrete logical gates used

in gradient descent) and can achieve very efficient inference (one million+ MNIST images/s on a single CPU core) [8]. Logical Neural Networks (LNNs) [9] map neural units to components of weighted logical formulas, yielding highly interpretable representations and supporting logic-consistent reasoning. LNNs use explicitly logic-structured model semantics. Differentiable Logic Machines (DLMs) [16] define a continuous relaxation of first-order logic program spaces (via predicate-weighted parameterization) and learn interpretable logic programs with gradient descent. In contrast to these approaches, which operate over structured relational inputs or operations on the input space, our method integrates logical composition in the feature space, it learns logical compositions over token features.

MoE architectures use a learned gating mechanism to route inputs to specialized expert subnetworks. In adaptive mixtures of local experts, the objective for each expert is decoupled from other experts, enabling specialization over different input regions, and thus allow both specialized and general networks [5]. These models can also improve inference efficiency via forced sparse activations. Conditional computation methods such as MoE and sparse routing motivates learned token-dependent routing decisions [10, 4]. Our model is closest to this line of work, but uses compact differentiable operators instead of expert-token dispatch among independent expert sub-networks.

Fuzzy-set theory provides continuous, differentiable functions [15]. We use fuzzy-set operations to estimate logical operations, which allows training with any off-the-shelf gradient-based optimizer. Our architecture is contrasted by Neural-Symbolic networks, where explicit logic is composed in the non-compressed input space (for example, de-

cision trees or markov-networks) [2].

3. Methods

3.1. Architecture and State Representation

The logic-augmented architecture introduces the logic-expert pathway parallel to the transformer backbone. At each layer t , the backbone produces hidden states:

$$H_t \in \mathbb{R}^{B \times S \times H}. \quad (1)$$

In parallel, the logic expert maintains logic states which are updated at every layer:

$$L_t \in \mathbb{R}^{B \times S \times L}, \quad (2)$$

At each layer, the logic state attends to the backbone via multi-head cross-attention, producing attended logic features:

$$\tilde{L}_t. \quad (3)$$

After the final layer, the logic state is fused with the backbone:

$$H_{\text{fused}} = H_{\text{last}} + \alpha W_f L_{\text{last}}, \quad (4)$$

where α controls the contribution of the logic pathway.

Notation. π_t : routing weights; s_t : gate activations; u_t : routed gate activations; B : batch size; S : sequence length; H : backbone hidden dimension; L : logic-state dimension; G : gates per operator family (AND/OR/NOT), with total routed channels $3G$; C_t : local window size at layer t ; τ : routing softmax temperature.

3.2. Cross-Attention into Logic Space

At each layer, logic tokens act as queries, while backbone hidden states are projected to keys and values. This produces attended logic features:

$$\tilde{L}_t := \text{CrossAttention}(L_{t-1}, H_t). \quad (5)$$

These features serve as the input to the routing and composition stage.

3.3. Routing and Gate Computation

Routing and gate activation use separate learned projection matrices: W_r produces routing logits, while W_g produces gate activations.

Routings are computed from $\tilde{L}_t$:

$$R_t := \tilde{L}_t W_r, \quad W_r \in \mathbb{R}^{L \times 3G}. \quad (6)$$

Routing weights are:

$$\pi_t[b, s, j] = \frac{\exp(R_t[b, s, j]/\tau)}{\sum_{k=1}^{3G} \exp(R_t[b, s, k]/\tau)}. \quad (7)$$

Gate activations are computed as:

$$s_t := \sigma(\tilde{L}_t W_g), \quad W_g \in \mathbb{R}^{L \times 3G}. \quad (8)$$

The routed soft activations are:

$$u_t^{\text{soft}} := \pi_t \odot s_t. \quad (9)$$

In soft mode, $u_t := u_t^{\text{soft}}$. In STE mode (forward binarized), we use

$$u_t^{\text{hard}} := \mathbf{1}[u_t^{\text{soft}} \geq 0.5], \quad u_t := u_t^{\text{soft}} + \text{sg}(u_t^{\text{hard}} - u_t^{\text{soft}}). \quad (10)$$

We split:

$$[u_t^{\text{AND}}, u_t^{\text{OR}}, u_t^{\text{NOT}}] := \text{split}(u_t; G, G, G). \quad (11)$$

3.4. Windowed Operator Composition

Let $v \in [0, 1]^{B \times S \times G}$ denote one operator group.

Windows are causal and defined along either: inter-token axis (sequence), or intra-token axis (channels).

For inter-token composition:

$$v_{\text{win}} \in \mathbb{R}^{B \times S \times G \times C_t}. \quad (12)$$

Operators are:

$$\text{AND}(x) = \prod_{i=1}^{C_t} x_i, \quad (13)$$

$$\text{OR}(x) = 1 - \prod_{i=1}^{C_t} (1 - x_i), \quad (14)$$

$$\text{NOT}(x) = 1 - x. \quad (15)$$

Applying to routed groups:

$$a_{b,s,g} := \text{AND}(\text{win}(u_t^{\text{AND}})_{b,s,g,:}), \quad (16)$$

$$o_{b,s,g} := \text{OR}(\text{win}(u_t^{\text{OR}})_{b,s,g,:}), \quad (17)$$

$$n_{b,s,g,:} := \text{NOT}(\text{win}(u_t^{\text{NOT}})_{b,s,g,:}). \quad (18)$$

AND and OR reduce to $[B, S, G]$.

NOT is applied elementwise and flattened to $[B, S, G \cdot C_t]$, to preserve negation operation structure.

The concatenated representation has dimension:

$$G + G + G \cdot C_t = G(C_t + 2). \quad (19)$$

3.5. Layer Update and Residual Flow

The concatenated operator outputs are projected:

$$\Delta L_t := \text{Linear}(\text{concat}(a, o, n)). \quad (20)$$

The logic state is updated via:

$$L_t = \text{RMSNorm}(L_{t-1} + \Delta L_t). \quad (21)$$

3.6. Output Fusion

After the final layer, fusion is applied as in Eq. 4. α may be fixed or learned, it directly controls the contribution of the logic expert.

4. Experiments

4.1. Setup

We use Llama-3.1-8B [7] as the shared backbone for all variants. We compare: (i) a Baseline model, (ii) a No-Gate Control (parameter-matched), and (iii+) the logic expert variants. The No-Gate Control serves as a parameter-matched control to separate routing effects from pure parameter-count gains. All variants use the 3 label classifier head. The original Llama head is not used. In this control, the parallel branch excludes cross-attention, operator routing, and compositional gating. All experiments use the AdamW [6] optimizer and cross-entropy loss, with full fine-tuning of the backbone in bfloat16. Every configuration is run with three seeds (42, 777, 123); the seed selects the training and validation subsets as well as the initialization, so comparisons between models are seed-paired. We call a difference supported only if its seed-paired mean is positive and exceeds two standard errors; the main hypothesis (routed inter-token $G = 8$ beats the No-Gate Control on ProofWriter) and this test were fixed before the reruns. Each run used one NVIDIA H100 80GB or H200 141GB GPU on the Oregon State University HPC cluster; the first version’s single runs used an NVIDIA RTX PRO 6000 Blackwell GPU on Google Colab.

4.2. Logical Reasoning – ProofWriter

We use the ProofWriter [11] dataset to compare logical reasoning to the baselines. ProofWriter is a logical reasoning dataset where each sample contains a small rulebase (facts plus rules) and a query to evaluate as true, false, or unknown. It is useful here because it directly tests multi-step compositional inference, which matches the intended behavior of the routed logic pathway. Each input is the rulebase followed by the question ("`{theory}`\nQuestion: `{question}`"), about 120 tokens on average, truncated from the left if needed so the question is always kept. We use deterministic, matched sampling across all models (10k train, 5k validation, all problem depths). Main runs are trained for 3 epochs; validation accuracy still rose from epoch 2 to 3 in 17 of 24 runs and fell by less than 1 pp in the rest, so longer training might raise scores slightly.

4.2.1 Beyond Logic – Multi-Genre Natural Language (MNLI)

To test performance beyond rulebase logic tasks, we evaluate performance on Multi-Genre Natural Language (MNLI) [14]. MNLI allow us to evaluate the natural language understanding of the model. MNLI has logical multi-class labels (Entailment, Neutral, Contradiction), this allows us to map to the same 3-label classification head used for ProofWriter.

4.3. Hyperparameter Evaluations

Primary hyperparameters are logic dimension: L , gate count: G , gate mode: (soft vs STE), gate window size: C_t , window axis: (inter-token vs intra-token), fusion α mode: (fixed vs learned), and initial fusion α . We validate on task validation sets, we use the No-Gate Control architecture to avoid attributing gains to unmatched model size.

4.4. Metrics

Training metrics are validation accuracy and validation loss. Operator mechanism metrics are routing entropy and fusion α . We report routing entropy normalized by its maximum, $H/\ln(3G)$, so that runs with different gate counts are comparable; 1 means uniform routing. We also report average gradient norm (Avg Grad Norm) as an optimization-health signal. Stable gradient norms generally indicate smoother updates, very large norms can indicate unstable training dynamics, small gradient norms indicates arrested training.

5. Results

5.1. ProofWriter

Table 1 reports final validation accuracy, validation loss, and mechanism metrics after 3 epochs. With the rulebase in the input, every model reaches 0.925–0.939 accuracy, so the backbone solves most of the task on its own. The No-Gate Control is the best model (0.939) and beats the Baseline by +1.4 pp (seed-paired SE 0.2 pp). Because the No-Gate Control also adds the fusion MLP and a normalized head, this gain should be attributed to that whole head path, not to parallel capacity alone.

No routed variant improves on the No-Gate Control. The seed-paired differences are -1.3 pp for inter-token $G = 8$ with $\alpha = 0.01$ (SE 0.8; per seed $-2.5, +0.3, -1.7$), -0.4 pp for intra-token $G = 8$ (SE 0.2), and -0.5 pp for $G = 16$ (SE 0.3). Changing α to 0.1 or learning it moves the mean by less than one seed standard deviation. Our main hypothesis, that inter-token routing beats the parameter-matched control, is therefore not supported. Inter-token and intra-token composition perform the same, and $G = 16$ is no worse than $G = 8$.

Gradient norms are lower for every model with the fusion head (19.6–22.9) than for the Baseline (28.5), with no

Model	Val Accuracy	Val Loss	Fusion α (final)	Norm. Routing Entropy	Grad Norm	First version
Baseline	0.925 ± 0.005	0.198 ± 0.003	–	–	28.5	0.417
No-Gate Control	0.939 ± 0.004	0.167 ± 0.014	0.01 (fixed)	–	20.3	0.525
Routed (intra, G=8)	0.936 ± 0.004	0.194 ± 0.023	0.01 (fixed)	0.972	19.6	0.542
Routed (inter, G=8)	0.927 ± 0.012	0.203 ± 0.021	0.01 (fixed)	0.972	22.9	0.562
Routed (inter, G=8)	0.936 ± 0.005	0.179 ± 0.021	0.1 (fixed)	0.972	22.2	0.546
Routed (inter, G=8)	0.938 ± 0.011	0.172 ± 0.007	0.0077 (learned from 0.01)	0.973	20.9	0.546
Routed (inter, G=16)	0.934 ± 0.002	0.198 ± 0.022	0.01 (fixed)	0.976	20.3	0.501
Routed (inter, G=8), no cross-attn	0.937 ± 0.010	0.178 ± 0.031	0.01 (fixed)	0.974	21.7	–

Table 1. ProofWriter validation results with the rulebase in the input, mean $\pm$ SD over 3 seeds (5,000 validation examples per seed). Normalized routing entropy is $H / \ln(3G)$ ($1 =$ uniform routing). Grad norm is the mean over the final epoch. “First version” is the single-run accuracy reported on 25 March 2026, which was measured on question-only inputs and is kept only for reference. Abbreviations: Baseline = backbone with a linear head on the last token (no parallel branch); No-Gate Control = parameter-matched parallel branch without cross-attention, operator routing, or compositional gating, with the same fusion and normalized head as the routed models; Routed = logic expert variants.

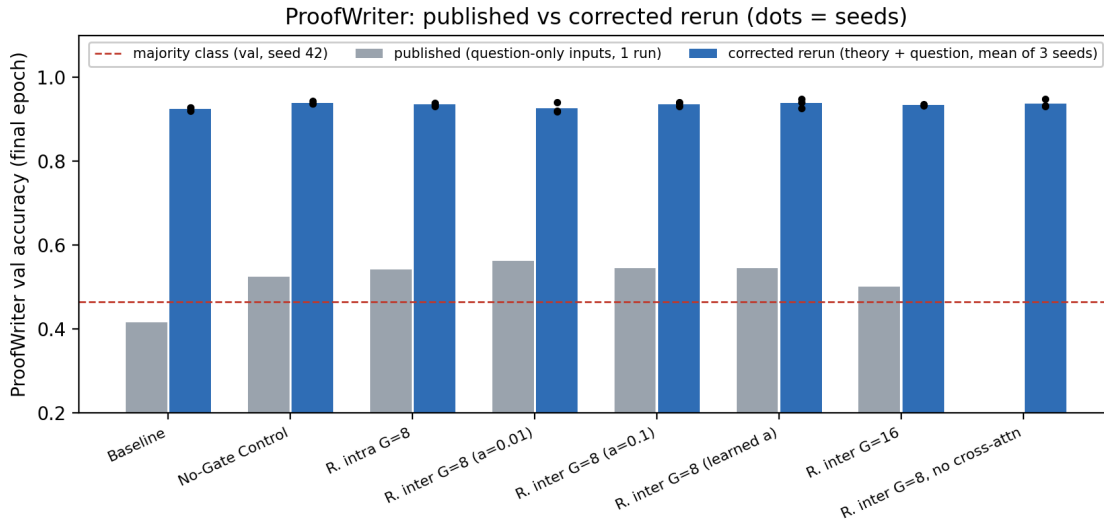


Figure 2. ProofWriter validation accuracy. Grey: single runs reported in the first version, measured on question-only inputs (the dashed line is the majority-class rate, 0.465). Blue: corrected reruns with the rulebase in the input, mean of 3 seeds; dots are individual seeds.

separation between routed and No-Gate models. Routing entropy is 97–98% of its maximum $\ln(3G)$ in every routed run, for both $G = 8$ and $G = 16$, so the router spreads its weight almost uniformly across operators (Section 6.2).

5.2. Size and Transfer

A key confound in models is added trainable parameters. Table 2 confirms close parameter matching between the No-Gate Control and the routed variants (within 0.05%). Since the routed variants do not outperform the control, parameter count does not hide a routing effect here. The routed model without cross-attention has 71M fewer parameters than the full routed model and performs the same (Section 6.1).

Model Variant	Trainable Parameters
Baseline	7,504,936,963
No-Gate Control	7,582,192,483
Routed (G=8)	7,579,068,163
Routed (G=16)	7,579,725,059
Routed (G=8), no cross-attn	7,507,724,035

Table 2. Trainable parameter counts. Inter- vs intra-token composition has no effect on the count. With fixed α the fusion weight is a buffer, so these counts are one lower than with learned α .

5.2.1 Multi-Genre Natural Language (MNLI)

MNLI [14]. combines sentence understanding with logic multi-class labels. We map the labels to the ProofWriter classes (important for the next test): contradiction $\rightarrow 0$, entailment $\rightarrow 1$, neutral $\rightarrow 2$. For comparability, evaluation uses matched sampling. Notable hyperparameters values in

the appendix.

Model	MNLI Val Acc	MNLI Val Loss	First version
No-Gate Control	0.871 $\pm$ 0.006	0.393 $\pm$ 0.058	0.888
Routed (intra, G=8)	0.860 $\pm$ 0.033	0.429 $\pm$ 0.114	0.880
Routed (inter, G=8)	0.851 $\pm$ 0.048	0.451 $\pm$ 0.132	0.878

Table 3. Performance on MNLI, mean $\pm$ SD over 3 seeds (500 validation examples per seed). “First version” is the single run reported on 25 March 2026; every first-version value lies within two standard deviations of the rerun mean.

Table 3 reports MNLI results. The No-Gate Control again has the highest mean accuracy (0.871). The routed variants are lower (seed-paired -1.1 pp intra, -2.0 pp inter) but not significantly (SE 1.6 and 2.5 pp), because the routed models vary more across seeds: one inter-token seed reaches only 0.796. The first version’s single-run values lie 1.7–2.7 pp above the three-seed means and within the seed spread. At this scale, routing gives no measurable transfer benefit on natural language inference.

5.3. Multi-task Training

For multi-task training, each mini-batch is sampled from a 50/50 mixture of ProofWriter and MNLI. This experiment tests whether routing can jointly optimize symbolic-style and open-domain NLI supervision without degrading either task. Both models use identical multi-task settings (listed in the appendix), and we report per-task validation metrics after joint training.

Model	ProofWriter Acc	MNLI Acc	Mean
No-Gate Control	0.918 $\pm$ 0.017	0.892 $\pm$ 0.012	0.905
Routed (inter, G=8)	0.937 $\pm$ 0.009	0.882 $\pm$ 0.016	0.909

Table 4. 50/50 mixed ProofWriter (with rulebase) and MNLI training, mean $\pm$ SD over 3 seeds. The first version reported 0.544 / 0.879 (No-Gate) and 0.536 / 0.881 (Routed) on question-only ProofWriter inputs.

Table 4 reports the 50/50 multi-task evaluation. Here the routed model is higher on ProofWriter ($+1.8$ pp seed-paired, SE 0.6) and lower on MNLI (-1.0 pp, SE 0.8), for about the same mean. The ProofWriter difference is the only comparison in this paper that favours routing, but it should be read as exploratory: it was not a pre-specified hypothesis, it is one of nine comparisons, and with three seeds a two-standard-error threshold is lenient (the 95% t critical value with 2 degrees of freedom is about 4.3; here $t \approx 3.2$). It could indicate that routing helps keep the logical task from being crowded out by joint training, which is worth testing with more seeds.

6. Discussion

With the rulebase in the input, a fully fine-tuned Llama-3.1-8B already reaches 0.925 on ProofWriter, and none of the logic-expert variants improves on a parameter-matched

Model	Validation Acc	Validation Loss
Routed (inter, G=8)	0.927 $\pm$ 0.012	0.203 $\pm$ 0.021
Routed (inter, G=8), no cross-attn	0.937 $\pm$ 0.010	0.178 $\pm$ 0.031
No-Gate Control	0.939 $\pm$ 0.004	0.167 $\pm$ 0.014

Table 5. ProofWriter with and without cross-attention into logic space, mean $\pm$ SD over 3 seeds. Without cross-attention the routed stream reads a linear projection of the backbone hidden state plus its previous state. This table replaces the first version’s comparison with an earlier implementation, whose No-Gate value (0.4646) equalled the majority-class rate of question-only inputs.

control. The mechanism metrics agree with the accuracy results. Routing entropy is near its maximum in every routed run, so the router does not specialize operators. When α is learned from 0.01 it shrinks to 0.0077 ± 0.0014 , so training turns the logic branch down rather than up. Together these suggest that at this scale the backbone does not need, and does not use, the operator pathway. The inter-token versus intra-token choice, which motivated this design, makes no measurable difference.

The only comparison that favours routing is ProofWriter accuracy under 50/50 multi-task training ($+1.8$ pp). It is exploratory and comes with an MNLI cost (-1.0 pp), but it is the natural place for a pre-registered follow-up with more seeds.

Fusion α needs care as a diagnostic. The fusion module is cast to the backbone’s bfloat16, where the spacing between representable values near 0.1 is about 4.9×10^{-4} . The α learning rate (2×10^{-4}) is below half of that, so every update rounds away and an α initialized at 0.1 never moves, even when it is nominally learned. This applies to all MNLI and multi-task runs. Near 0.01 the spacing is small enough for α to move, which is why the learned- α ProofWriter run shows a real trajectory.

6.1. Cross-Attention Ablation

Table 5 removes cross-attention from the routed model while keeping everything else fixed. The model without cross-attention scores $+1.1$ pp higher (seed-paired SE 1.1 pp, not significant) with 71M fewer parameters, so cross-attention into logic space contributes nothing measurable on this task.

6.2. Routing Visualization

Figure 3 shows the routing weights of the corrected inter-token model. Most cells are close to the uniform value $1/24$, with isolated cells up to about 0.17 and no consistent operator preference across layers. The first version read its heatmap as sparse with a few dominant operators. Normalized entropy of 0.97–0.98 shows that this reading does not hold: the router stays close to uniform.

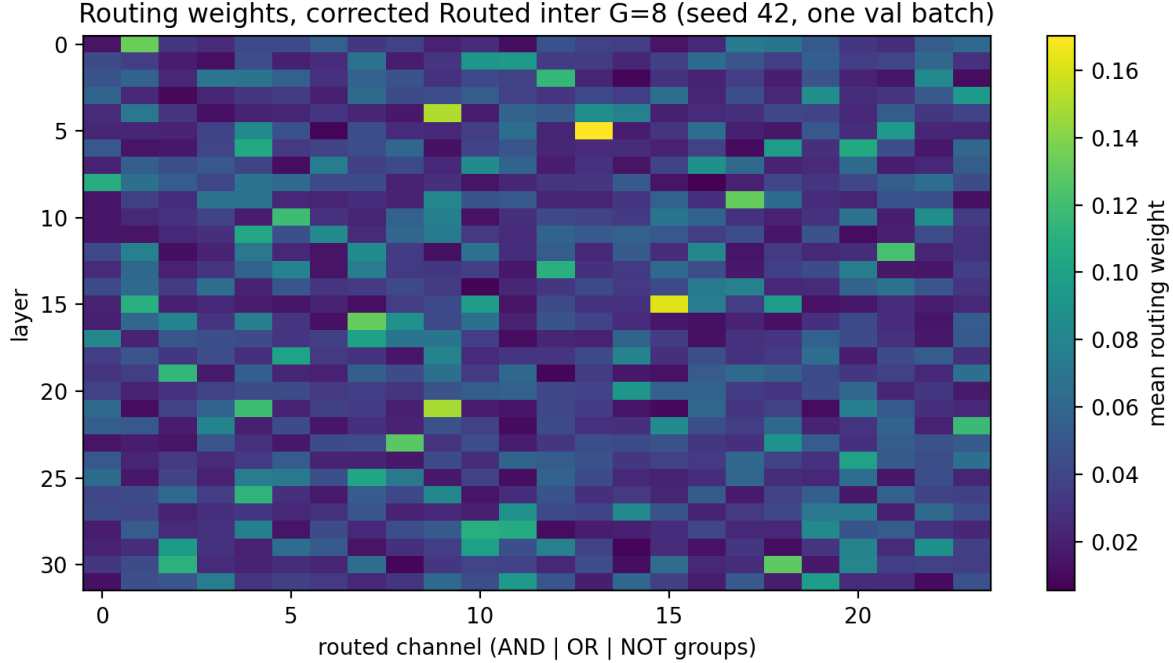


Figure 3. Routing-weight heatmap over transformer depth for the corrected Routed (inter, $G = 8$) model (seed 42, one validation batch, averaged over tokens). Rows are layers and columns are the $3G = 24$ routed channels (AND, OR, NOT groups); color is mean routing weight. Uniform routing is $1/24 \approx 0.042$, which is where most cells sit.

7. Limitations

We evaluate one logical reasoning benchmark and one transfer task, with a single 8B backbone and three seeds per configuration. ProofWriter with the rulebase is largely solved by the backbone alone (0.925), which leaves little headroom for an auxiliary pathway; harder or out-of-distribution reasoning splits could behave differently. All models train in pure bfloat16 without fp32 master weights, which rounds away small updates (see α above) and may under-train the logic modules in general. We kept the original hyperparameters and did not retune any variant. The routing design also introduces hyperparameters (gate count, window size, routing temperature) whose interactions are not fully explored.

8. Conclusion

This architecture shifts conditional computation from routing among independent neural experts to routing over fuzzy logical operator groups, with matched-capacity controls and mechanism-level diagnostics. In seed-paired comparisons on ProofWriter with the rulebase in the input, no routed variant outperforms a parameter-matched No-Gate Control, cross-attention into logic space adds nothing measurable, and the inter-token versus intra-token choice does not matter. On MNLI, routed and No-Gate variants are

also indistinguishable. Near-maximal routing entropy and a shrinking learned α indicate that the model barely uses or specializes the operator pathway. The positive ProofWriter result in the first version of this paper came from a data pipeline that dropped the rulebase and from an input-blind Baseline. Useful next steps are harder reasoning benchmarks where the backbone is not already near ceiling, fp32 training of the logic modules, and a pre-registered multi-task test of the one exploratory gain we observed.

References

- [1] I. Aleksander, M. D. Gregorio, F. M. G. França, P. M. V. Lima, and H. Morton. A brief introduction to weightless neural systems. In *The European Symposium on Artificial Neural Networks*, 2009.
- [2] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha. Neural-symbolic learning and reasoning: A survey and interpretation, 2017.
- [3] DeepSeek-R1 Team. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, Sept. 2025.
- [4] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

- [5] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [6] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.
- [7] Meta AI. The llama 3 herd of models, 2024.
- [8] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen. Deep differentiable logic gate networks, 2022.
- [9] R. Riegel, A. Gray, F. Luus, N. Khan, N. Makondo, I. Y. Akhalwaya, H. Qian, R. Fagin, F. Barahona, U. Sharma, S. Ikbal, H. Karanam, S. Neelam, A. Likhyan, and S. Srivastava. Logical neural networks, 2020.
- [10] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.
- [11] O. Tafjord, B. D. Mishra, and P. Clark. Proofwriter: Generating implications, proofs, and abductive statements over natural language, 2021.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [14] A. Williams, N. Nangia, and S. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122. Association for Computational Linguistics, 2018.
- [15] L. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.
- [16] M. Zimmer, X. Feng, C. Glanois, Z. Jiang, J. Zhang, P. Weng, D. Li, J. Hao, and W. Liu. Differentiable logic machines, 2023.

Appendix

A. ProofWriter Evaluation Details

This appendix summarizes the main ProofWriter configuration used in the experiments.

A.1. Settings and Hyperparameters

- Batch size: 30 (train), 30 (eval)
- Epochs: 3
- Learning rate: 1×10^{-5} (fusion α : 2×10^{-4} when learned)
- Weight decay: 0.01
- Logic dimension / gates / heads: 256 / 8 (16) / 4
- Gate mode: `soft`

- Precision: `bfloat16`, gradient checkpointing on
- Input: `rulebase (theory)` then question, left-truncated, at most 250 tokens
- Seeds: 42, 777, 123

A.2. Dataset(s)

- ProofWriter Depth: `all`: (all allowed)
- Train samples: 10,000
- Validation samples: 5,000

B. MNLI Evaluation Details

For MNLI transfer, labels are remapped into the shared three-class order used throughout this paper: contradiction $\rightarrow$ 0, entailment $\rightarrow$ 1, and neutral $\rightarrow$ 2.

B.1. Dataset(s)

- Train samples: 5,000 (`train split`)
- Validation samples: 500 (`validation matched`)

B.2. Settings and Hyperparameters

- Train batch size: 20
- Eval batch size: 10
- Epochs: 2
- Learning rate: 1×10^{-5}
- Weight decay: 0.01
- Logic dimension / gates / heads: 256 / 8 / 4
- Fusion α : 0.1, nominally learned but frozen by `bfloat16` rounding
- Gate mode: `soft`
- Seeds: 42, 777, 123

C. Multi-task Evaluation Details

For multi-task experiments, each batch is sampled from a mix of ProofWriter and MNLI datasets.

C.1. Dataset(s)

- MNLI sampling ratio: 0.5
- ProofWriter sampling ratio: 0.5
- ProofWriter depth: `all`
- Train/validation samples: 10,000 / 1,000

C.2. Settings and Hyperparameters

- Train batch size: 20
- Eval batch size: 100
- Epochs: 6
- Learning rate: 1×10^{-5}
- Weight decay: 0.01
- Logic dimension / gates / heads: 256 / 8 / 4
- Fusion α : 0.1, nominally learned but frozen by bfloat16 rounding
- Gradient checkpointing: on; seeds 42, 777, 123
- Gate axis: `inter_token`
- Gate mode: `soft`

D. Cross-Attention Ablation Details

Table 5 uses the ProofWriter settings above with inter-token routing, $G = 8$, and fixed $\alpha = 0.01$. In the ablated model no cross-attention modules are built; at each layer the routed stream takes the logic projection of the backbone hidden state plus the previous logic state, the same input path as the No-Gate Control.

E. Changes from the First Version

- The ProofWriter loader rendered only the question, because the dataset source stores facts and rules in a `theory` column that the loader did not read. All ProofWriter results are rerun with the rulebase.
- The Baseline pooled position 0, which in a causal model is the BOS token and carries no input information; it now pools the last non-padding token, like the other models.
- The first version listed $\alpha = 0.00001$ for several rows; the logged configurations used 0.01. MNLI used 5,000/500 examples with batch size 20, not 4,000/200 with 30.
- The earlier-implementation comparison (Routed-Gate-Only vs No-Gate) is replaced by a controlled cross-attention ablation.
- Single runs are replaced by three seeds with a pre-specified comparison rule.

F. ProofWriter Question

This appendix section provides an example question from ProofWriter.

F.1. Rulebase

```
triple1: The cow is big.
triple2: The cow needs the dog.
triple3: The dog sees the rabbit.
triple4: The rabbit chases the cow.
triple5: The rabbit chases the dog.
triple6: The rabbit is big.
triple7: The rabbit sees the dog.
rule1: If the cow is blue and the cow needs the rab
rule2: If the cow chases the dog then the cow sees
rule3: If something is big then it chases the dog.
```

F.2. Question, Answer

```
Question: The cow sees the rabbit?
Answer: true
```